Obesity Classification Project Report

Subject: Machine Learning

International Institute of Information Technology, Bangalore

Team Members:

1. Nitheesh Vemula, MT2025079
2. Guduru Veera Siva Somanath, MT2025048

GitHub Repository

Contents

Abstract

This project predicts a person's weight category (underweight, normal, overweight, or obese) using information such as age, gender, height, weight, family history, and lifestyle habits. Several machine learning models were tested, and the XGBoost classifier gave the best results. The project demonstrates how data analysis, feature engineering, and model tuning can accurately classify a person's weight category.

1 Introduction

This project builds a machine learning model to predict a person's Weight Category using different personal and lifestyle details. The dataset includes information such as Age, Gender, Height, Weight, Family Health History, Physical Activity, and Eating Habits. The process follows a clear step-by-step data science workflow — starting with data exploration and cleaning, then moving to feature engineering, encoding categorical data, and handling outliers.

Several machine learning models were tested, including K-Nearest Neighbors (KNN), Decision Tree, Random Forest, Gradient Boosting, and XGBoost. These models were compared using Stratified K-Fold Cross-Validation and evaluated on Accuracy, F1 Score, Precision, and Recall. After trying different hyperparameter tuning methods (GridSearchCV), the XGBoost Classifier gave the best and most consistent performance.

Finally, the optimized XGBoost model was trained on the entire dataset and used to predict the test data.

2 Exploratory Data Analysis

We looked at the data to see how personal and lifestyle factors affect a person's weight category. The following bar charts show the main findings.

- **Weight Categorization:**
  Different output Classes

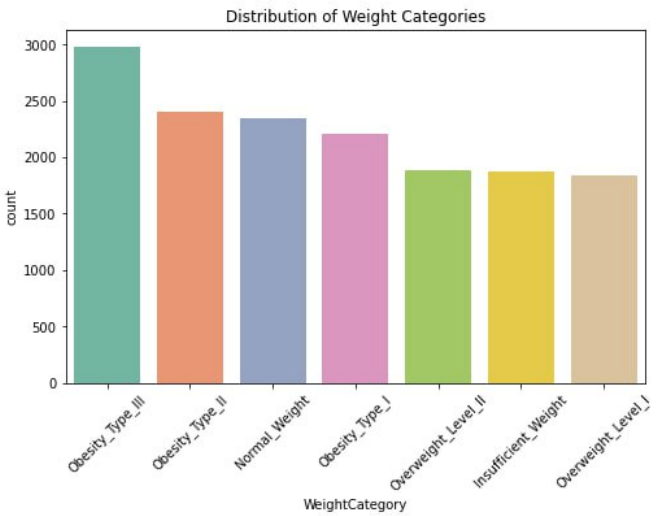

Figure 1: Weights classes

- **Data Description:** Understanding data

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
id	15533.0	NaN	NaN	NaN	7766.0	4484.135201	0.0	3883.0	7766.0	11649.0	15532.0
Gender	15533	2	Male	7783	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Age	15533.0	NaN	NaN	NaN	23.816308	5.663167	14.0	20.0	22.771612	26.0	61.0
Height	15533.0	NaN	NaN	NaN	1.699918	0.08767	1.45	1.630927	1.7	1.762921	1.975663
Weight	15533.0	NaN	NaN	NaN	87.785225	26.369144	39.0	66.0	84.0	111.600553	165.057269
family_history_with_overweight	15533	2	yes	12696	NaN	NaN	NaN	NaN	NaN	NaN	NaN
FAVC	15533	2	yes	14184	NaN	NaN	NaN	NaN	NaN	NaN	NaN
FCVC	15533.0	NaN	NaN	NaN	2.442917	0.530895	1.0	2.0	2.34222	3.0	3.0
NCP	15533.0	NaN	NaN	NaN	2.760425	0.706463	1.0	3.0	3.0	3.0	4.0
CAEC	15533	4	Sometimes	13126	NaN	NaN	NaN	NaN	NaN	NaN	NaN
SMOKE	15533	2	no	15356	NaN	NaN	NaN	NaN	NaN	NaN	NaN
CH2O	15533.0	NaN	NaN	NaN	2.027626	0.607733	1.0	1.796257	2.0	2.531456	3.0
SCC	15533	2	no	15019	NaN	NaN	NaN	NaN	NaN	NaN	NaN
FAF	15533.0	NaN	NaN	NaN	0.976968	0.836841	0.0	0.00705	1.0	1.582675	3.0
TUE	15533.0	NaN	NaN	NaN	0.613813	0.602223	0.0	0.0	0.566353	1.0	2.0
CALC	15533	3	Sometimes	11285	NaN	NaN	NaN	NaN	NaN	NaN	NaN
MTRANS	15533	5	Public_Transportation	12470	NaN	NaN	NaN	NaN	NaN	NaN	NaN
WeightCategory	15533	7	Obesity_Type_III	2983	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Figure 2: Data description

- **Gender Distribution:** Males and females show different trends in weight categories.

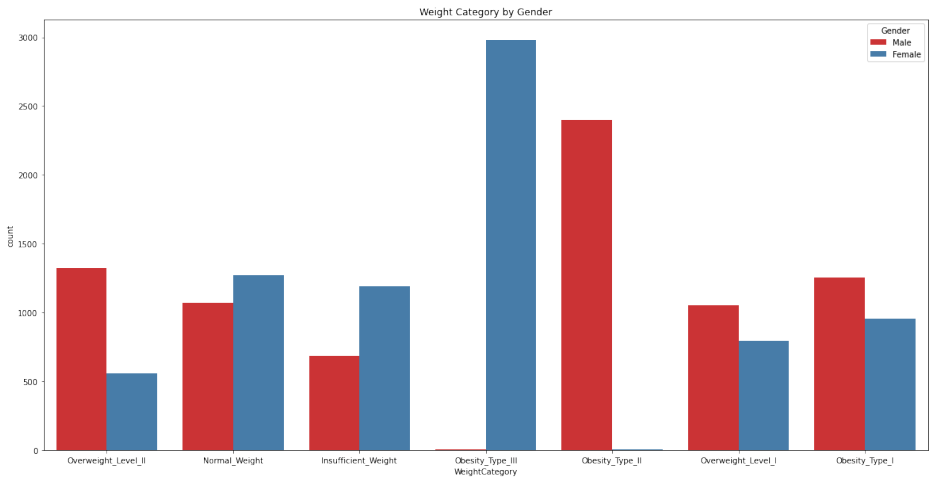

Figure 3: Gender Analysis

- **Family History:** People with family history of overweight show more obesity cases.

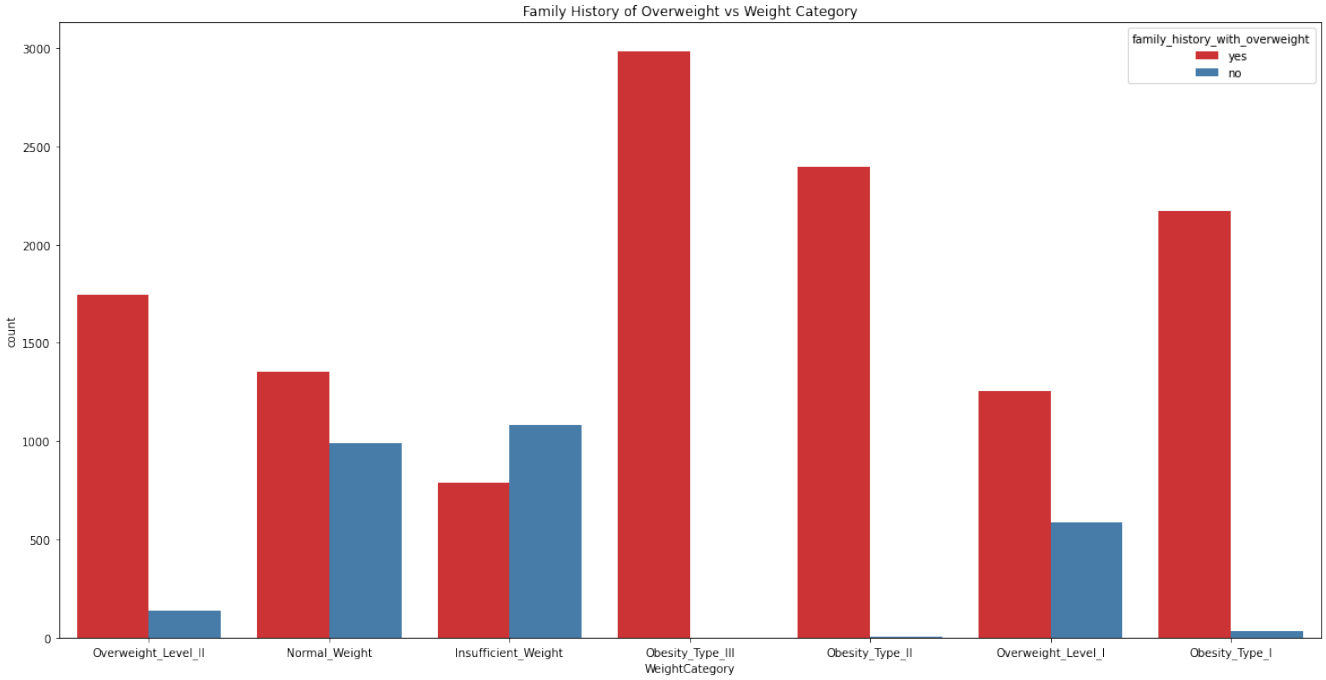

Figure 4: Family History Impact

- **Age Analysis:** Age analysis on data

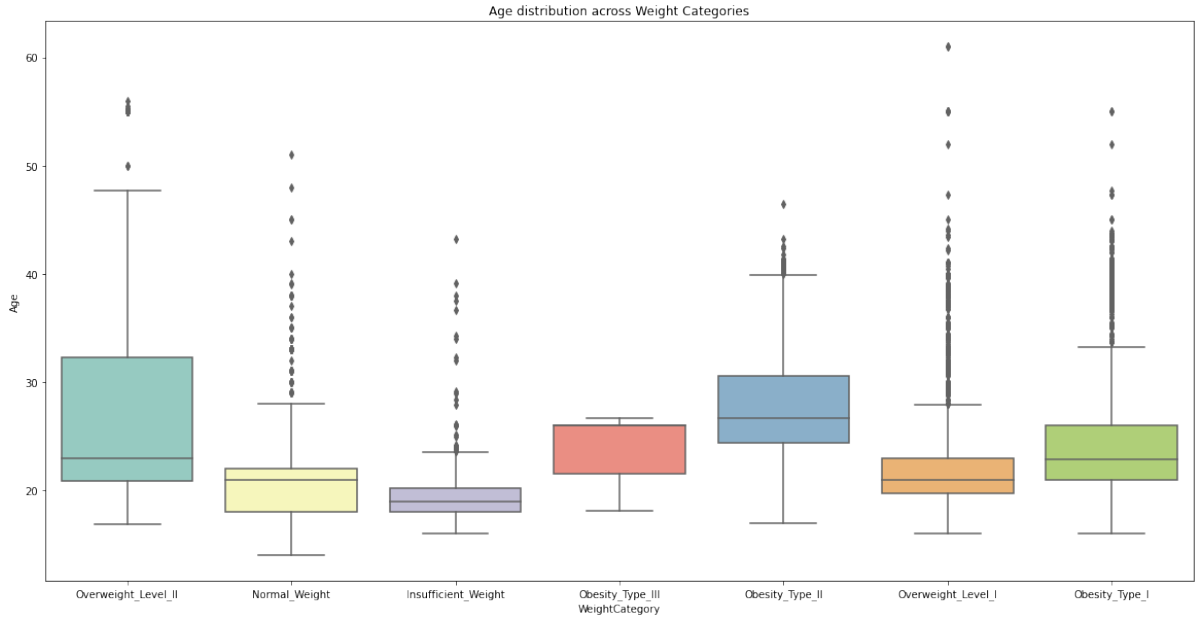

Figure 5: Age wise categorization

- **Height vs Weight Analysis:** Height Weight analysis on data

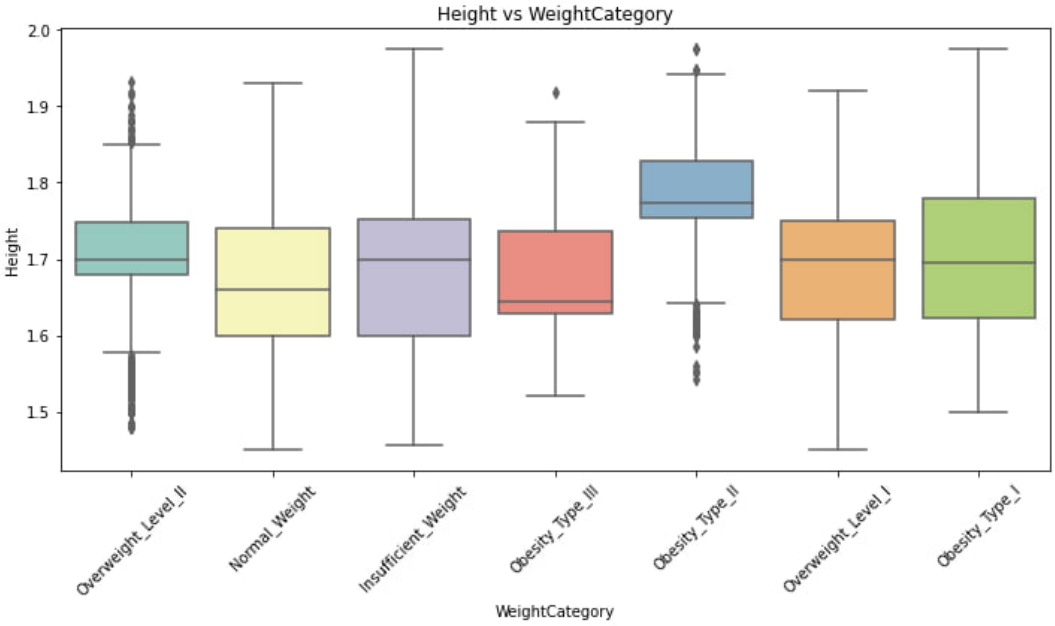

Figure 6: Height Weight Scatterplot according to classification

- **Correlation Analysis:** Correlation between numeric features.We analyzed how the features in the dataset relate to each other using a correlation map. Key observations:
  - Features which show a moderate positive correlation, meaning as one increases, the other tends to increase too.
  - Features which are weakly correlated or not correlated, meaning they provide different information for the model.
  - Understanding these relationships helps in feature selection and avoids redundant information.
  - We observed that there is very less corelation in between columns

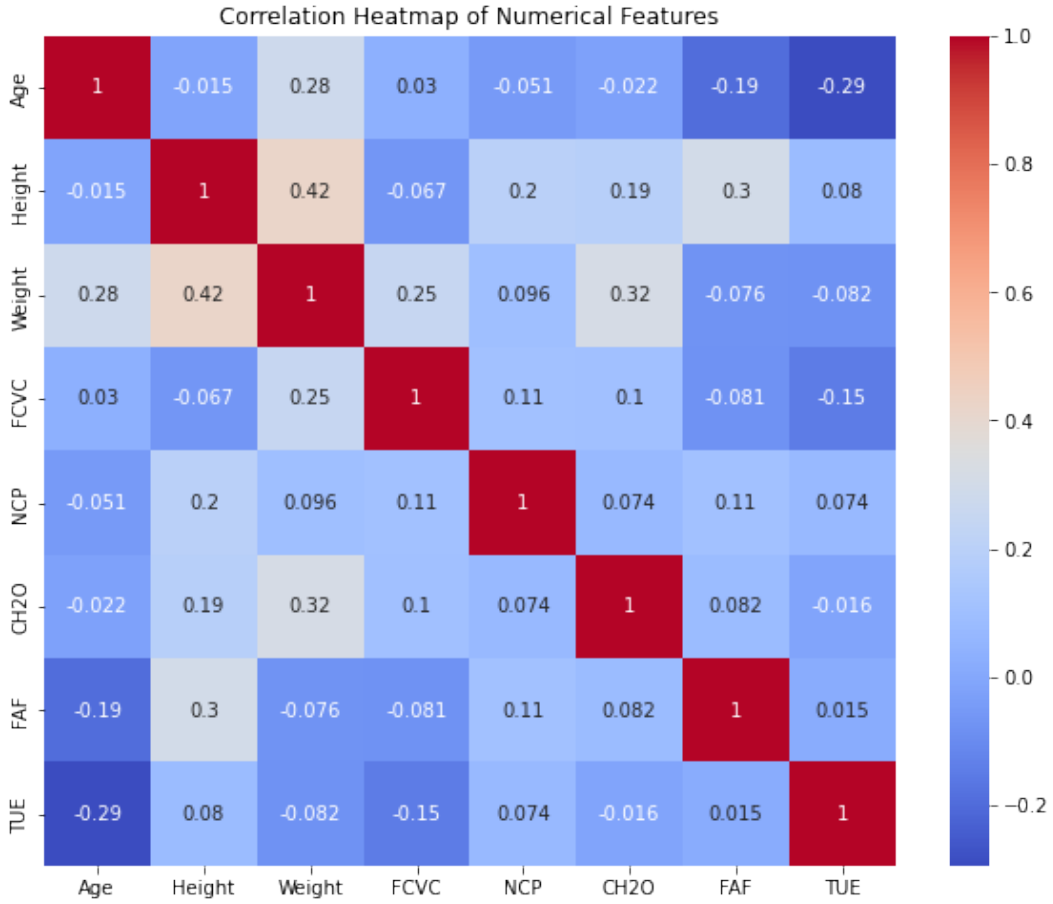

Figure 7: Correlation map

- **Distributions of different features:**
- We observed the different features distributions for scaling and processing

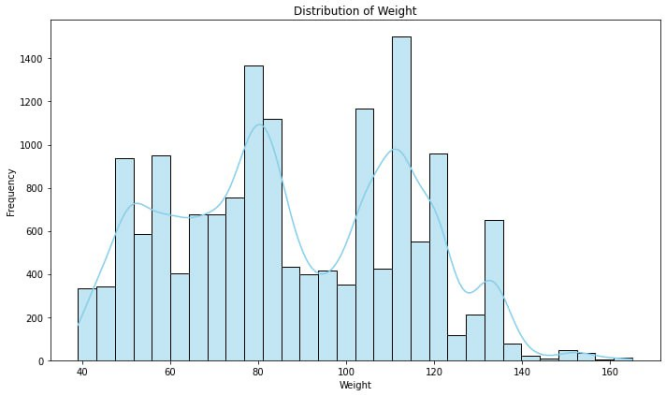

Figure 8: Weight class distribution

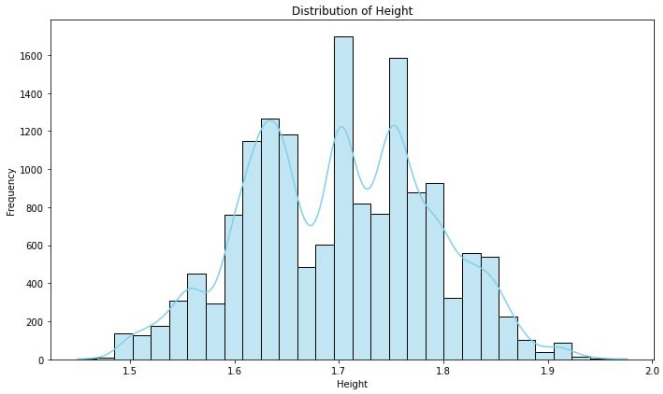

Figure 9: Height class distribution

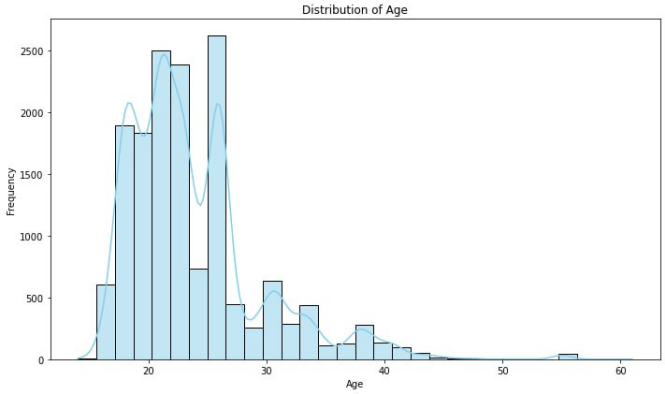

Figure 10: Age class distribution

3 Data Processing

The dataset contains both categorical and numerical features. To prepare it for machine learning models, the following steps were performed:

- **Checking for NULL and Missing Values:** All features were inspected for missing or null values. No missing or null values were found. Even though we implemented the median,mode replacement for numerical missing data. If there exists some invalid values in numerical columns such height,age,weight we handle such cases by replacing with median values
- **Encoding Categorical Variables:** Features such as Gender, FAVC (frequent consumption of high-calorie foods), and CAEC (caloric consumption during meals) are categorical. These were converted into numerical form using *Label Encoding*, where each category is replaced with a unique integer value so that machine learning models can process them.
- **Normalizing Numeric Features:** Continuous features like `Age`, `Height`, and `Weight` were scaled to a similar range. Normalization ensures that features with larger numeric ranges do not dominate those with smaller ranges during model training, improving model performance and convergence.
- **Train-Test Split:** The dataset was divided into training and testing sets with an 80:20 ratio. The training set is used to train the models, while the testing set evaluates model performance on unseen data. A fixed `random_state` was used to ensure reproducibility of the split.

4 Feature Engineering

To enhance the predictive power of the models, several new features were created based on existing columns:

- **BMI (Body Mass Index):**
$$\text{BMI} = \frac{\text{Weight}}{\text{Height}^2}$$
Standardizes body fat based on height and weight.
- **Weight-to-Age Ratio:**
$$\text{weight\_age\_ratio} = \frac{\text{Weight}}{\text{Age}}$$
Shows how weight varies relative to age.
- **Height-to-Age Ratio:**
$$\text{height\_age\_ratio} = \frac{\text{Height}}{\text{Age}}$$
Captures how height relates to age.
- **Food Consumption per Physical Activity (FCVC per FAF):**
$$\text{FCVC\_per\_FAF} = \frac{\text{FCVC}}{\text{FAF} + 1 \times 10^{-5}}$$
Represents food consumption relative to physical activity.

- **Number of Meals per Time Using Electronic Devices (NCP per TUE):**

$$\text{NCP\_per\_TUE} = \frac{\text{NCP}}{\text{TUE} + 1 \times 10^{-5}}$$

Measures meal frequency relative to sedentary behavior.

- **Water Intake per Weight:**

$$\text{Water\_per\_Weight} = \frac{\text{CH2O}}{\text{Weight}}$$

Normalizes water consumption by body weight.

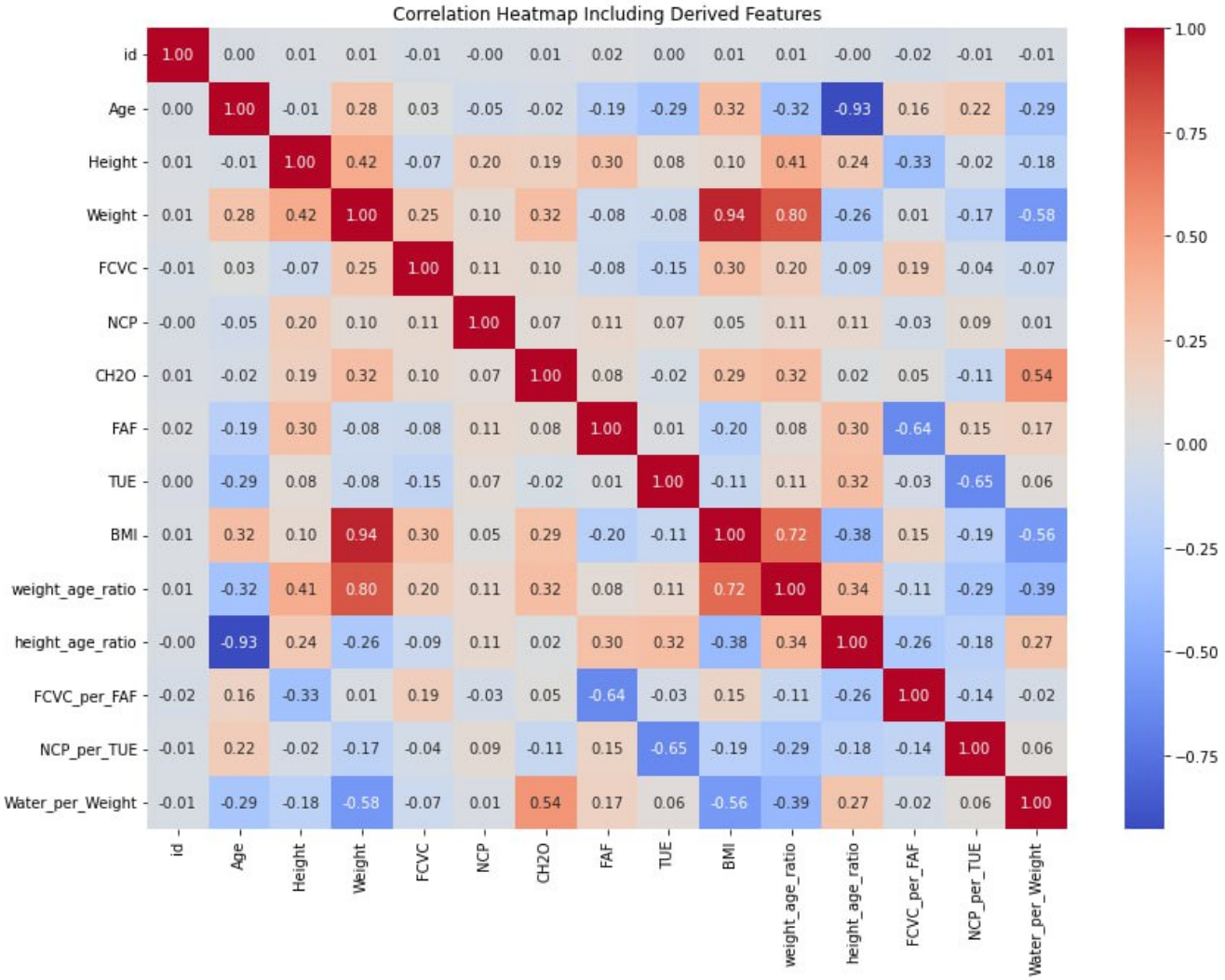

Figure 11: Correlation Matrix of Numerical Features After Feature Engineering

- **Observations:**
  - `BMI` has a strong positive correlation with `WeightCategory`.
  - `weight_age_ratio` and `height_age_ratio` show moderate correlations with the target.
  - Features like `FCVC_per_FAF`, `NCP_per_TUE`, and `Water_per_Weight` may help the model capture finer patterns.

5 Models Used

The following models were used:

- **KNN algorithm**
- **Decision Tree**
- **Random forest classifier**
- **Gradient Boosting**
- **XGBoost Classifier**

6 Hyperparameter Tuning

Models were tuned using Grid Search with 5-fold cross-validation.

- **Decision tree:**
  - `min_samples_split = [2,5,10]`
  - `max_depth = [None,5,10,15,20]`
  - `min_samples_leaf = [1,2,5]`
- **Random Forest:**
  - `n_estimators = [100,200,300]`
  - `min_samples_split = [2,5,10]`
  - `max_depth = [None,5,10,15,20]`
  - `min_samples_leaf = [1,2,5]`
  - `max_features = [sqrt,log2]`
- **XGBoost Classifier:**
  - **max_depth:** 4 - 10

  - **n_estimators:** 500 - 2000
  - **gamma:** float between 0 and 1
  - **reg_alpha:** float between 0 and 1
  - **reg_lambda:** float between 0 and 1
  - **min_child_weight:** integer between 0 and 10
  - **subsample:** float between 0 and 1
  - **colsample_bytree:** float between 0 and 1
  - **learning_rate:** float between 0 and 1

- **Gradient Classifier:**
  - `learning_rate = [0.01,0.05,0.1]`
  - `min_samples_split = [2,5,10]`
  - `n_estimators = [200,300,400]`
  - `max_depth = [3,5,7]`
  - `min_samples_leaf = [1,2,5]`
  - `subsample = [0.8,1.0]`

Best Parameters (XGBoost Classifier):

- `max_depth = 10`
- `n_estimators = 1224`
- `gamma = 0.8323`
- `reg_alpha = 0.9212`
- `reg_lambda = 0.8523`
- `min_child_weight = 4`
- `subsample = 0.9233`
- `colsample_bytree = 0.4785`
- `learning_rate = 0.06436`
- `eval_metric = 'mlogloss'`

Model	Accuracy	Precision	Recall	F1-score
KNN	0.759	0.752	0.759	0.753
Decision Tree	0.873	0.873	0.873	0.872
Random Forest	0.894	0.894	0.894	0.893
Gradient Boosting	0.899	0.899	0.900	0.899
XGBoost	0.907	0.907	0.907	0.907

Table 1: Performance metrics for different classifiers

7 Performance

The models were evaluated using accuracy, precision, recall, and F1-score.

Observations:

- XGBoost performed slightly better than Gradient and random forest.
- KNN had lower performance.
- The feature importance analysis showed that Gender,AGE,Weight,Height were the most influential characteristics.
- **Comparsion Plot of different algorithms:**

  This shows the comparison between different algorithms based on there performance on test data

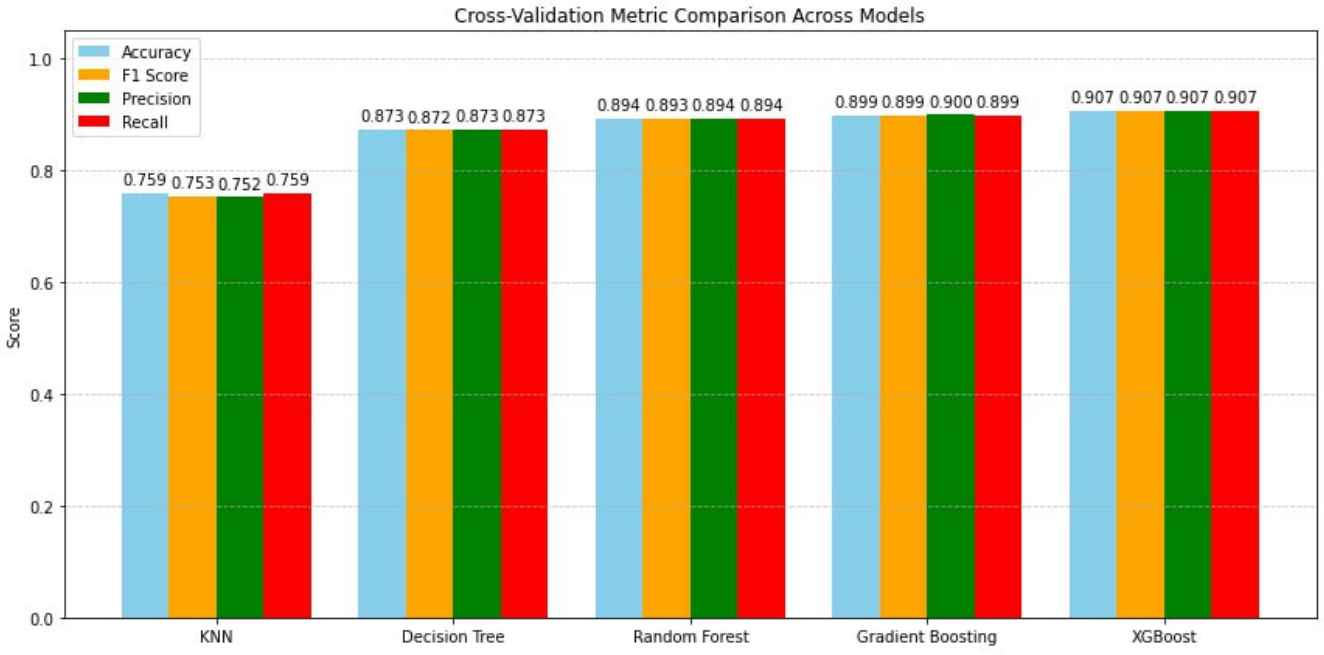

Figure 12: Comparison

- **Confusion Matrix of KNN:**

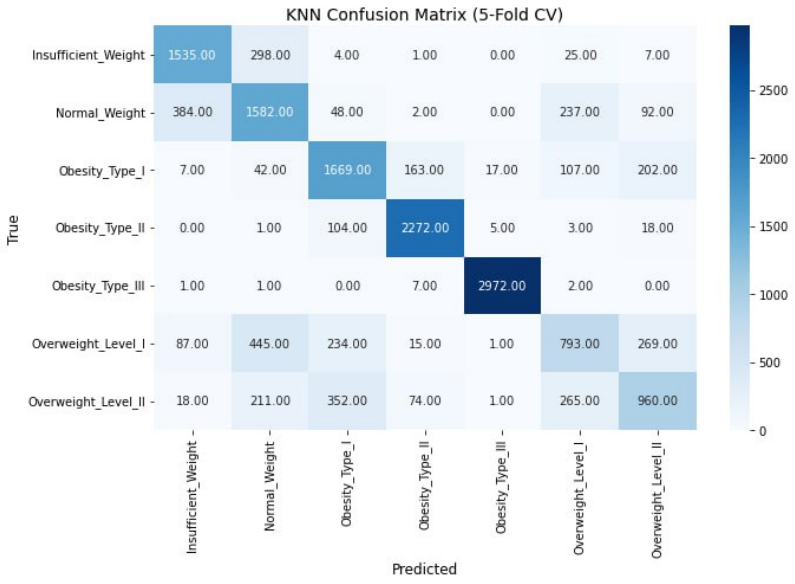

Figure 13: Confusion Matrix

- **Confusion Matrix of Decision tree:**

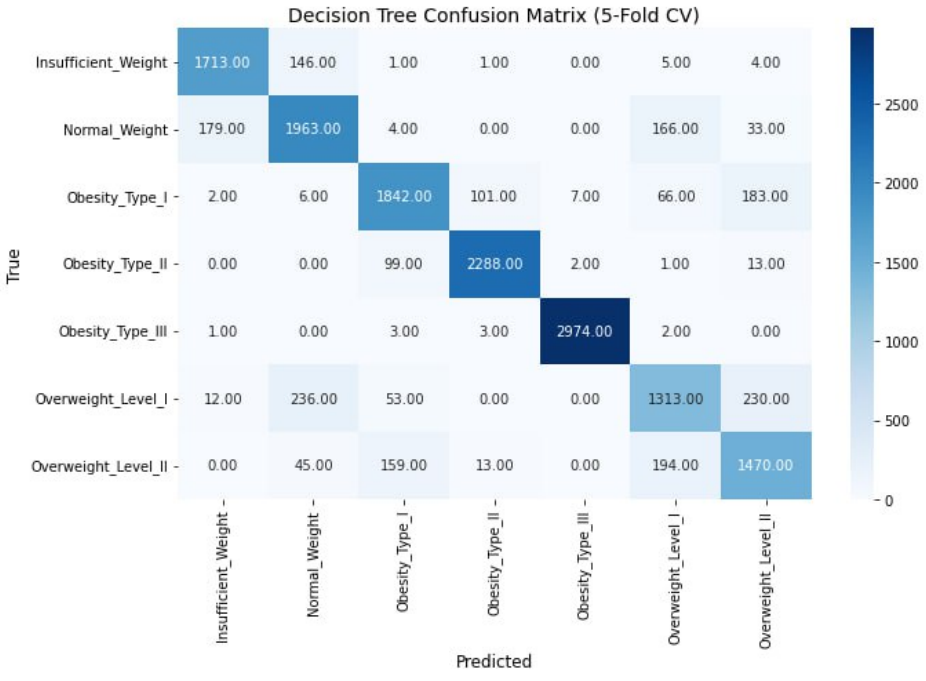

Figure 14: Confusion Matrix

- **Confusion Matrix of Random forest tree:**

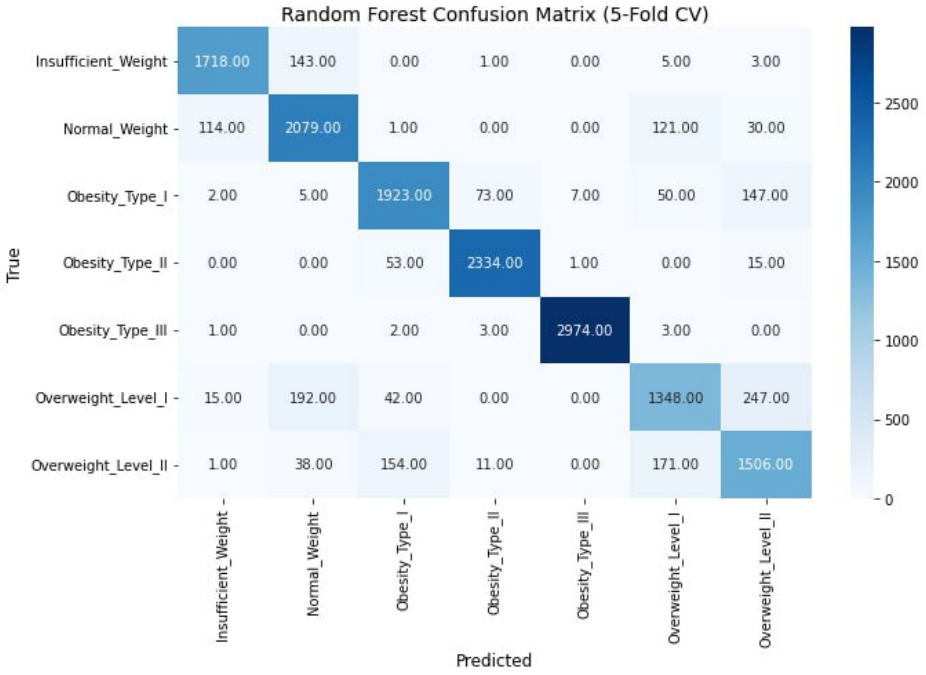

Figure 15: Confusion Matrix

- **Confusion Matrix of Gradient Boosting:**

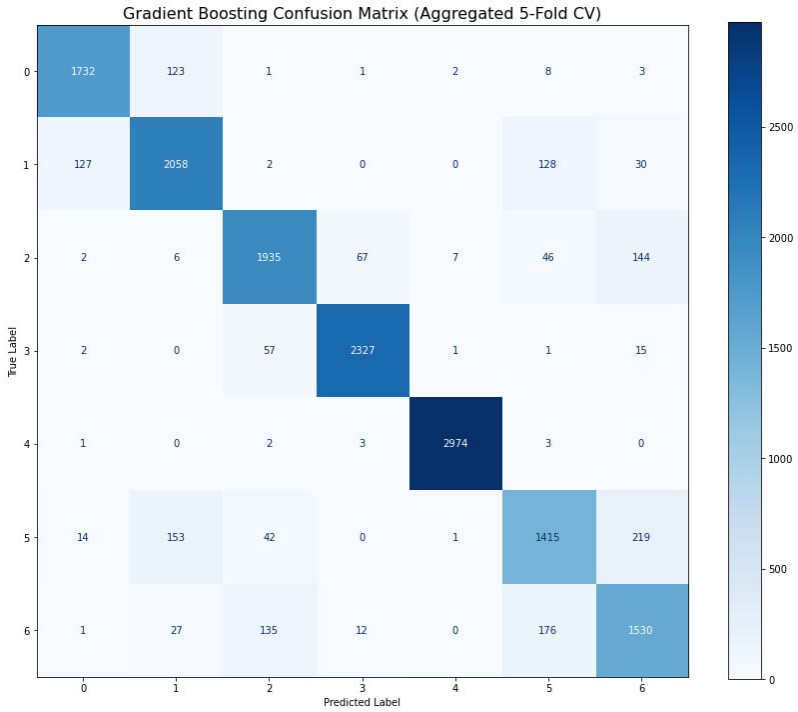

Figure 16: Confusion Matrix

- **XGBOOST confusion matrix:**

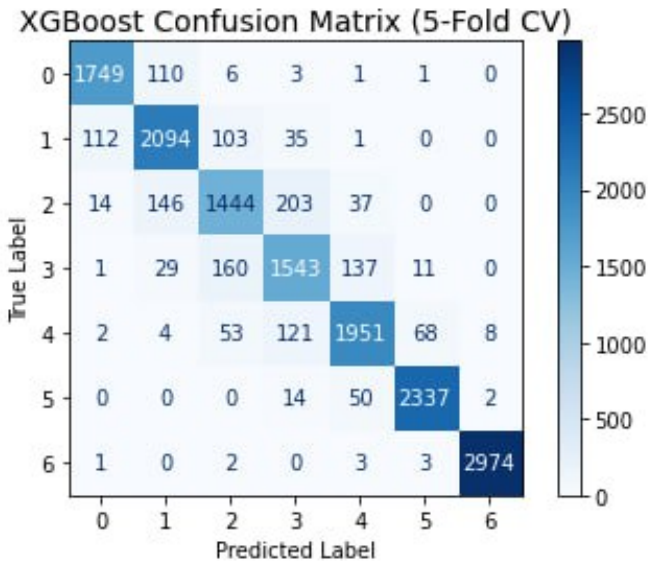

Figure 17: Confusion Matrix

- **Feature Importance Plotting**

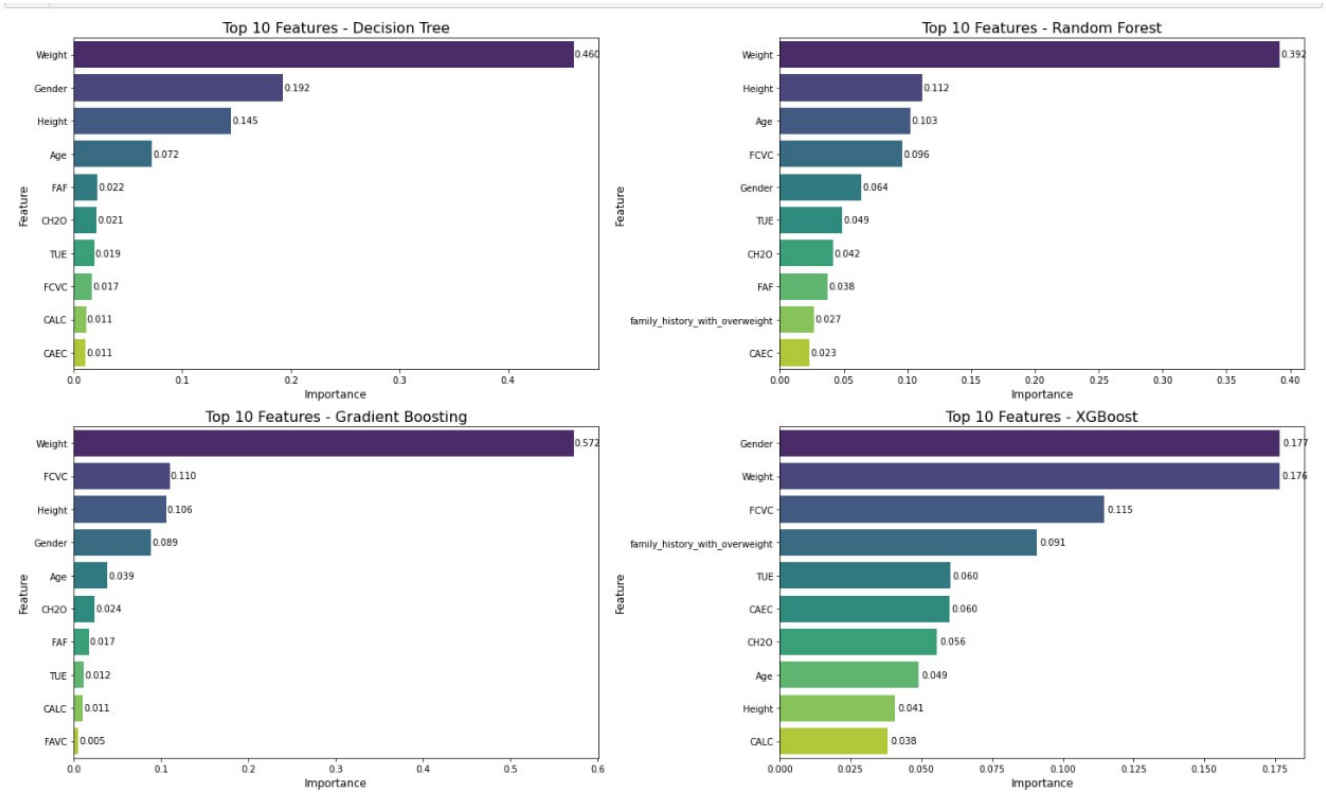

Figure 18: Feature Importance Plots

8 Code Snippets

```
import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedKFold

from sklearn.metrics import accuracy_score,
f1_score,precision_score, recall_score, confusion_matrix,
ConfusionMatrixDisplay

from xgboost import XGBClassifier
import matplotlib.pyplot as plt


df = pd.read_csv('train.csv', index_col=0)

df_test = pd.read_csv("test.csv", index_col=0)


features = df.drop('WeightCategory', axis=1)
labels = df['WeightCategory']

mask_numeric = features.dtypes == float
df_numerical = features.loc[:, mask_numeric]

mask_categorical = features.dtypes != float
df_categorical = features.loc[:, mask_categorical]

df_encoded = df_categorical.copy(deep=True)
df_encoded['Gender'] = df_categorical['Gender'].map({'Male': 0,
'Female': 1})

df_encoded['family_history_with_overweight'] =
df_categorical ['family_history_with _overweight']
.map({'no': 0,

'yes': 1})
df_encoded['FAVC'] = df_categorical['FAVC'].map({'no': 0, 'yes':
1})
df_encoded['CAEC'] = df_categorical['CAEC'].map({'no': 0,
'Sometimes': 1, 'Frequently': 2, 'Always': 3})

df_encoded['SMOKE'] = df_categorical['SMOKE'].map({'no': 0,
'yes': 1})
df_encoded['SCC'] = df_categorical['SCC'].map({'no': 0, 'yes':
```

```
1})
df_encoded['CALC'] = df_categorical['CALC'].map({'no': 0,
'Sometimes': 1, 'Frequently': 2, 'Always': 3})


df_onehot = pd.get_dummies(df_categorical['MTRANS']).astype(int)

df_encoded.drop('MTRANS', axis=1, inplace=True)
df_encoded = pd.concat([df_encoded, df_onehot.iloc[:, :-1]],
axis=1)

df_all_features = pd.concat([df_numerical, df_encoded], axis=1)


def apply_preprocessing(data):
    features = data.copy(deep=True)
    mask_numeric = features.dtypes == float
    df_numerical = features.loc[:, mask_numeric]
    mask_categorical = features.dtypes != float
    df_categorical = features.loc[:, mask_categorical]

    df_encoded = df_categorical.copy(deep=True)
    df_encoded['Gender'] = df_categorical['Gender'].map({'Male':
    0, 'Female': 1})
    df_encoded['family_history_with_overweight'] =
    df_categorical['family_history_with_overweight'].
    map({'no':
    0, 'yes': 1})
    df_encoded['FAVC'] = df_categorical['FAVC'].map({'no': 0,
    'yes': 1})
    df_encoded['CAEC'] = df_categorical['CAEC'].map({'no': 0,
    'Sometimes': 1, 'Frequently': 2, 'Always': 3})
    df_encoded['SMOKE'] = df_categorical['SMOKE'].map({'no': 0,
    'yes': 1})
    df_encoded['SCC'] = df_categorical['SCC'].map({'no': 0,
    'yes': 1})
    df_encoded['CALC'] = df_categorical['CALC'].map({'no': 0,
    'Sometimes': 1, 'Frequently': 2, 'Always': 3})

    df_onehot = pd.get_dummies(df_categorical['MTRANS']).astype(int)

    df_encoded.drop('MTRANS', axis=1, inplace=True)

    df_encoded = pd.concat([df_encoded, df_onehot.iloc[:, :-1]],
    axis=1)
```

```
    df_all_features = pd.concat([df_numerical, df_encoded],
    axis=1)

    return df_all_features

df_test = apply_preprocessing(df_test)

dict_conversion = {
    'Insufficient_Weight': 0, 'Normal_Weight': 1,
    'Overweight_Level_I': 2,

    'Overweight_Level_II': 3, 'Obesity_Type_I': 4,
    'Obesity_Type_II': 5, 'Obesity_Type_III': 6

}

reverse_dict_conversion = {v: k for k, v in
dict_conversion.items()}

y = labels.map(dict_conversion).values

X = df_all_features

best_params_XGB = {
    'max_depth': 10, 'n_estimators': 1224, 'gamma': 0.8323,
    'reg_alpha': 0.9212,

    'reg_lambda': 0.8523, 'min_child_weight': 4, 'subsample':
    0.9233,
    'colsample_bytree': 0.4785, 'learning_rate': 0.06436,
    'eval_metric': 'mlogloss'
}

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)


accuracy_scores = []
f1_scores = []
precision_scores = []
recall_scores = []

y_true_all = []
y_pred_all = []
for fold, (train_idx, val_idx) in enumerate(skf.split(X, y), 1):
```

```
    X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
    y_train, y_val = y[train_idx], y[val_idx]

    clf = XGBClassifier(**best_params_XGB, random_state=42,
    n_jobs=-1, use_label_encoder=False)
    clf.fit(X_train, y_train)

    y_pred = clf.predict(X_val)

    accuracy_scores.append(accuracy_score(y_val, y_pred))
    f1_scores.append(f1_score(y_val, y_pred, average='weighted'))

    precision_scores.append(precision_score(y_val, y_pred,
    average='weighted'))
    recall_scores.append(recall_score(y_val, y_pred,
    average='weighted'))

    y_true_all.extend(y_val)
    y_pred_all.extend(y_pred)


clf_final = XGBClassifier(**best_params_XGB, random_state=42,
n_jobs=-1, use_label_encoder=False)
clf_final.fit(X, y)

predictions = clf_final.predict(df_test)
predictions_labels = [reverse_dict_conversion[p] for p in predictions]

df_submission = pd.read_csv("test.csv")
df_submission['WeightCategory'] = predictions_labels
df_submission[['id',
'WeightCategory']].to_csv('BestSolution.csv', index=False)
```

9 Conclusion

In this project, we built a model to predict a person's Weight Category using features like age, gender, height, weight, and lifestyle habits. We tried several machine learning models, including KNN, Decision Tree, Random Forest, Gradient Boosting, and XGBoost.

The XGBoost model gave the best results, achieving 91.07% accuracy on the test data. The most important features for predicting Weight Category were **Weight**, **Height**, and **Gender**.

This shows that with proper data processing, feature engineering, and tuning, we can accurately predict weight categories. Future work could include more features to improve accuracy even further.